Figure 11.5 UI on Remix IDE for the DCC.sol smart contract

Observe that the `loginStudent()` function is like a gatekeeper that allows only registered students to add courses. It is a `view` function, so it is not recorded on the chain. It might be interesting for audit purposes to keep a trace of login activities. In that case, you can remove the keyword view from the function header definition.

11.6 Local deployment

The next step in the roadmap is navigating from the Remix IDE to test the DCC.sol on the local Ganache test chain. By now, you should be familiar with the sequence of these steps. Click the Ganache icon, and click Quickstart. Wait for it to start. Download the codebase for DCC-Dapp-local.zip from the chapter's codebase. Unzip it. Then navigate to DCC-contract.

The commands

```
cd DCC-contract
truffle migrate --reset
```

compile and deploy the contracts in the contracts directory. There are two contracts: DCC.sol and Migrations.sol. As they are compiled and deployed, you'll see several messages, ending with the final message that two contracts were deployed, as shown in figure 11.6. These messages indicate that two smart contracts--DCC.sol and Migrations.sol--were deployed successfully. Recall that the migration process itself is written as a Solidity smart contract. The next step is testing this DCC.sol deployed on the Ganache test chain.